

**INSTITUTE OF ENGINEERING AND TECHNOLOGY
DEEN DAYAL UPADHYAYA GORAKHPUR UNIVERSITY
GORAKHPUR**



**COURSE STRUCTURE
&
SYLLABUS**

FOR

B.TECH.

in

Artificial Intelligence and Machine Learning

Based

on

AICTE MODEL CURRICULUM

[Effective from the Session: 2025-26]

Branch/Course: Artificial Intelligence and Machine Learning

Semester-III (Second Year) Curriculum

S. No.	Type of Course	Course Code	Course Title	Credit
1.	Professional Core Course	AIML201	Artificial Intelligence	3+0
2.	Professional Core Course	AIML202	Data Structure	4+0
3.	Professional Core Course	AIML203	Programming in Python	2+0
4.	Professional Core Course	AIML204	Data Analytics using R Programming	3+0
5.	Professional Core Course	AIML205	Mathematical Logic	3+0
6.	Professional Core Course	AIML251	Artificial Intelligence Lab	0+1
7.	Professional Core Course	AIML252	Data Structure Lab	0+1
8.	Professional Core Course	AIML253	Python Programming Lab	0+1
9.	Professional Core Course	AIML254	R-Programming Lab	0+1
10.	SEC Course	SEC-x	SEC Course	03
11.	AEC Course	AEC-x	AEC Course	02
Total Credit				24

Artificial Intelligence (AIML201)**Credit: 3+0**

Course Objective: This course sheds light on the fundamentals of Artificial Intelligence and its applications in various areas. The student will learn to apply knowledge representation techniques and problem-solving strategies to common AI applications.

Syllabus

- 1. Introduction:** Definition – Future of Artificial Intelligence – Characteristics of Intelligent Agents– Typical Intelligent Agents – Problem Solving Approach to Typical AI problems.
- 2. Problem Solving Methods:** Problem solving Methods – Search Strategies- Uninformed – Informed – Heuristics – Local Search Algorithms and Optimization Problems – Searching with Partial Observations – Constraint Satisfaction Problems – Constraint Propagation – Backtracking Search – Game Playing – Optimal Decisions in Games – Alpha – Beta Pruning – Stochastic Games.
- 3. Knowledge Representation:** First Order Predicate Logic – Prolog Programming – Unification – Forward Chaining-Backward Chaining – Resolution – Knowledge Representation – Ontological Engineering-Categories and Objects – Events – Mental Events and Mental Objects – Reasoning Systems for Categories – Reasoning with Default Information.
- 4. Software Agents & Applications:** Architecture for Intelligent Agents – Agent communication – Negotiation and Bargaining – Argumentation among Agents – Trust and Reputation in Multi- agent systems. **Applications:** AI applications – Language Models – Information Retrieval- Information Extraction – Natural Language Processing – Machine Translation – Speech Recognition – Robot – Hardware – Perception – Planning – Moving.

Course Outcome (CO): At the end of course, the student will be able

CO1: To understand the basics of the theory and practice of Artificial Intelligence as a discipline and about intelligent agents.

CO2: Understand search techniques and gaming theory.

CO3: The student will learn to apply knowledge representation techniques and problem-solving strategies to common AI applications.

CO4: Students should be aware of techniques used for classification and clustering.

References

1. S. Russell and P. Norvig, "Artificial Intelligence: A Modern Approach", Prentice Hall, Third Edition, 2009.
2. Bratko, "Prolog: Programming for Artificial Intelligence", Fourth edition, Addison-Wesley Educational Publishers Inc., 2011.
3. M. Tim Jones, –Artificial Intelligence: A Systems Approach, Jones and Bartlett Publishers, Inc. First Edition, 2008
4. Nils J. Nilsson, –The Quest for Artificial Intelligence, Cambridge University Press, 2009.
5. William F. Clocksin and Christopher S. Mellish, Programming in Prolog: Using the ISO

Artificial Intelligence Lab (AIML251)

Credit: 0+1

1. Write a program to conduct uninformed and informed search.
2. Write a program to conduct game search.
3. Write a program to construct a Bayesian network from given data.
4. Write a program to infer from the Bayesian network.
5. Write a program to run value and policy iteration in a grid world.
6. Write a program to do reinforcement learning in a grid world
7. Implementation of toy problems
8. Developing agent programs for real world problems
9. Implementation of constraint satisfaction problems
10. Implementation and Analysis of DFS and BFS for an application
11. 11. Developing Best first search and A* Algorithm for real world problems
12. Implementation of minimax algorithm for an application
13. Implementation of unification and resolution for real world problems.
14. Implementation of knowledge representation schemes - use cases
15. Implementation of uncertain methods for an application
16. Implementation of block world problem

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner.

Course Objective: To provide the knowledge of basic data structures and its implementations in context of writing efficient programs.

Syllabus

- 1. Introduction:** Basic Terminology, Elementary Data Organization, Built in Data Types in C. Algorithm, Efficiency of an Algorithm, Time and Space Complexity, Asymptotic notations: Big Oh, Big Theta and Big Omega, Time-Space trade-off. Abstract Data Types (ADT) Arrays: Definition, Single and Multidimensional Arrays, Representation of Arrays: Row Major Order, and Column Major Order, Derivation of Index Formulae for 1-D, 2-D, 3-D and n-D Array Application of arrays, Sparse Matrices and their representations. Linked lists: Array Implementation and Pointer Implementation of Singly Linked Lists, Doubly Linked List, Circularly Linked List, Operations on a Linked List. Insertion, Deletion, Traversal, Polynomial Representation and Addition Subtraction & Multiplications of Single variable & two variables Polynomial.
- 2. Stacks and Queues:** Abstract Data Type, Primitive Stack operations: Push & Pop, Array and Linked Implementation of Stack in C, Application of stack: Prefix and Postfix Expressions, Evaluation of postfix expression, Iteration and Recursion- Principles of recursion, Tail recursion, Removal of recursion Problem solving using iteration and recursion with examples such as binary search, Fibonacci numbers, and Hanoi towers. Tradeoffs between iteration and recursion. Queues: Operations on Queue: Create, Add, Delete, Full and Empty, Circular queues, Array and linked implementation of queues in C, Dequeue and Priority Queue.
- 3. Searching & Sorting:** Concept of Searching, Sequential search, Index Sequential Search, Binary Search. Concept of Hashing & Collision resolution Techniques used in Hashing. Sorting: Insertion Sort, Selection, Bubble Sort, Quick Sort, Merge Sort, Heap Sort and Radix Sort.
- 4. Trees & Graphs:** Basic terminology used with Tree, Binary Trees, Binary Tree Representation: Array Representation and Pointer (Linked List) Representation, Binary Search Tree, Strictly Binary Tree, Complete Binary Tree. Extended Binary Trees, Tree Traversal algorithms: Inorder, Preorder and Postorder, Constructing Binary Tree from given Tree Traversal, Operation of Insertion, Deletion, Searching & Modification of data in Binary Search. Threaded Binary trees, Traversing Threaded Binary trees. Huffman coding using Binary Tree. Concept & Basic Operations for AVL Tree, B Tree & Binary Heaps. Graphs: Terminology used with Graph, Data Structure for Graph Representations: Adjacency Matrices, Adjacency List, Adjacency. Graph Traversal: Depth First Search and Breadth First Search, Connected Component, Spanning Trees, Minimum Cost Spanning Trees: Prim's and Kruskal algorithm. Transitive Closure and Shortest Path algorithm: Warshall Algorithm and Dijkstra Algorithm.

Course Outcome (CO): At the end of course, the student will be able to

Institute of Engineering and Technology, DDUGU, Gorakhpur

CO1: Describe how arrays, linked lists, stacks, queues, trees, and graphs are represented in memory, used by the algorithms and their common applications.

CO2: Discuss the computational efficiency of the sorting and searching algorithms.

CO3: Implementation of Trees and Graphs and perform various operations on these data structures.

CO4: Understanding the concept of recursion, application of recursion and its implementation and removal of recursion.

References

1. Aaron M. Tenenbaum, Yedidya Langsam and Moshe J. Augenstein, "Data Structures Using C and C++", PHI Learning Private Limited, Delhi India.
2. Horowitz and Sahani, "Fundamentals of Data Structures", Galgotia Publications Pvt Ltd Delhi India.
3. Lipschutz, "Data Structures" Schaum's Outline Series, Tata McGraw-hill Education (India) Pvt. Ltd.
4. Thareja, "Data Structure Using C" Oxford Higher Education.
5. AK Sharma, "Data Structure Using C", Pearson Education India.
6. Rajesh K. Shukla, "Data Structure Using C and C++" Wiley Dreamtech Publication.
7. Michael T. Goodrich, Roberto Tamassia, David M. Mount "Data Structures and Algorithms in C++", Wiley India.
8. P. S. Deshpandey, "C and Data structure", Wiley Dreamtech Publication.
9. R. Kruse et al, "Data Structures and Program Design in C", Pearson Education.
10. Berziss, AT: Data structures, Theory and Practice, Academic Press.
11. Jean Paul Trembley and Paul G. Sorenson, "An Introduction to Data Structures with Applications", McGraw Hill.
12. Adam Drozdek "Data Structures and Algorithms in Java", Cengage Learning.

Data Structure Lab (AIML252)

Credit: 0+1

1. Write C Programs to illustrate the concept of the following:
2. Sorting Algorithms-Non-Recursive.
3. Sorting Algorithms-Recursive.
4. Searching Algorithms.
5. Implementation of Stack using Array.
6. Implementation of Queue using Array.
7. Implementation of Circular Queue using Array.
8. Implementation of Stack using Linked List.
9. Implementation of Queue using Linked List.
10. Implementation of Circular Queue using Linked List.
11. Implementation of Tree Structures, Binary Tree, Tree Traversal, Binary Search Tree, Insertion and Deletion in BST.
12. Graph Implementation, BFS, DFS, Minimum cost spanning tree, shortest path algorithm

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner.

Programming in Python (AIML203)

Credit: 2+0

Course Objectives: This course enables students to solve problems in Python.

Syllabus

1. **Programming Basics and Decision Making :** Key features and applications of Python, Python Editors and Compilers (Interpreters), Using different offline and online Python IDE, Interacting with Python programs, Data types: Numeric, Boolean, Strings, Lists, Sets, Tuples, Dictionary; Variables: Declaration and initialization; Other concepts: Operators, Expressions, Indentation, Comments, Casting, Simple Statements: Taking inputs from user, Displaying outputs; Conditional statements: If...Else. Introduction to Linux: Understanding Linux, Installation of Python on Linux
2. **Control Flow and Other Programming Concepts** For Loops, While Loops, Break, Continue; Array: Looping Array elements, Array methods; Functions: Local and Global Variables, Built-in functions, User defined functions, Declaration of a function, Defining the function, Calling of the function, Functions with arguments, Recursion.
3. **OOP and File Handling:** Classes and objects, attributes and methods, constructors and destructors, inheritance, polymorphism, Exception Handling: Try...Except; Management of text files: Type of files, various file operations on text files, creating a text file, opening a file, closing a file, reading a text file, writing into a text file, copying a file to another file.
4. **Advance Concepts:** Problem solving- Use of Python to solve real time problems, How Python helps to research problems, Creating various types of graphs corresponding to any data to show different kinds of results and analysis; Data Analysis: Understanding problems of data science and machine learning, Creating codes in Python for various data analysis problems, Other advance programs.

Course Outcomes:

CO1: Examine Python syntax and semantics and be fluent in the use of Python flow control and functions.

CO2: Demonstrate proficiency in handling Strings and File Systems.

CO3: Create, run and manipulate Python Programs using core data structures like Lists, Dictionaries and use Regular Expressions.

CO4: Interpret the concepts of Object-Oriented Programming as used in Python.

References

1. Core Python Programming, Wesley J. Chun, Second Edition, Pearson.
2. Think Python, Allen Downey, Green Tea Press.
3. Introduction to Python, Kenneth A. Lambert, Cengage
4. Python Programming: A Modern Approach, Vamsi Kurama, Pearson
5. Learning Python, Mark Lutz, O'Really
6. Allen B. Downey, "Think Python: How to Think Like a Computer Scientist ", 2nd edition, Updated for Python 3, Shroff/O'Reilly Publishers, 2016
(<http://greenteapress.com/wp/thinkpython/>)
7. Guido van Rossum and Fred L. Drake Jr, —An Introduction to Python – Revised and updated

Python Programming Lab (AIML253)**Credit: 0+1**

1. Krishna and his five friends have decided to go for an industrial visit by sharing the expenses of the fuel equally. Write a Python program to calculate the amount (in Rs) each of them needs to put in for the complete journey. The program should also display **True**, if the amount to be paid by each person is divisible by 3, otherwise it should display **False**.
Hint: Use the relational operators in the print statement.
Assumptions: Assume that mileage of the vehicle, amount per litre of fuel and distance for one way are given.
2. Write a python program to find the best of two test average marks out of three test's marks accepted from the user.
3. Develop a Python program to check whether a given number is palindrome or not and also count the number of occurrences of each digit in the input number.
4. Defined as a function F as $F_n = F_{n-1} + F_{n-2}$. Write a Python program which accepts a value for N (where $N > 0$) as input and pass this value to the function. Display a suitable error message if the condition for input value is not followed.
5. Develop a Python program to convert binary to decimal, octal to hexadecimal using functions.
6. Write a Python program that accepts a sentence and find the number of words, digits, uppercase letters and lowercase letters.
7. Write a Python program to implement insertion sort and merge sort using lists.
8. Write a program to convert roman numbers into integer values using dictionaries.
9. Write a function called `is_phonenumber()` to recognize a pattern 415-555-4242 without using regular expression and also write the code to recognize the same pattern using regular expression.
10. Develop a Python program that could search the text in a file for phone numbers (+919900889977) and email addresses (sample@gmail.com).
11. Write a Python program to accept a file name from the user and perform the following operations
 - a. Display the first N line of the file.
 - b. Find the frequency of occurrence of the word accepted from the user in the file.
12. Write a Python program to create a ZIP file of a particular folder which contains several files inside it.

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner.

Data Analytics using R Programming (AIML204)

Credit: 3+0

Course Objective: The objective of this course is to teach students how to think logically about data and analyze it. The course stresses on analytics of data and reasoning and describes different ways in which problems could be solved.

Syllabus

1. Introduction to Data Analytics: Sources and nature of data, classification of data, characteristics of data, introduction to Big Data platform, need of data Analytics, evolution of analytic scalability, analytic process and tools, analysis vs reporting, modern Data analytic tools, applications of data analytics. Data Analytics Lifecycle: Need, key roles for successful analytic projects, various phases of data Analytics lifecycle – discovery, data preparation, model planning, model building, communicating Results, operationalization.
2. Data Analysis: Regression modeling, multivariate analysis, Bayesian modeling, inference and Bayesian networks, support vector and kernel methods, analysis of time series: linear systems Analysis & nonlinear dynamics, rule induction, neural networks: learning and generalisation, Competitive learning, principal component analysis and neural networks, fuzzy logic: extracting Fuzzy models from data, fuzzy decision trees, stochastic search methods.
3. Mining Data Streams: Introduction to streams concepts, stream data model and architecture, stream Computing, sampling data in a stream, filtering streams, counting distinct elements in a stream, Estimating moments, counting oneness in a window, decaying window, Real-time Analytics Platform (RTAP) applications, Case studies – real time sentiment analysis, stock market predictions.
4. Frequent Item sets and Clustering: Mining frequent item sets, market-based modelling, Apriori Algorithm, handling large data sets in main memory, limited pass algorithm, counting frequent item Sets in a stream, clustering techniques: hierarchical, K-means, clustering high dimensional data, CLIQUE and ProCLUS, frequent pattern-based clustering methods, clustering in non-Euclidean Space, clustering for streams and parallelism.

References and References:

1. Michael Berthold, David J. Hand, Intelligent Data Analysis, Springer
2. Anand Rajaraman and Jeffrey David Ullman, Mining of Massive Datasets, Cambridge University Press.
3. Bill Franks, Taming the Big Data Tidal wave: Finding Opportunities in Huge Data Streams with Advanced Analytics, John Wiley & Sons.
4. John Garrett, Data Analytics for IT Networks: Developing Innovative Use Cases, Pearson Education
5. Michael Minelli, Michelle Chambers, and Ambiga Dhiraj, "Big Data, Big Analytics: Emerging Business Intelligence and Analytic Trends for Today's Businesses", Wiley

R Programming Lab (AIML254)

Credit: 0+1

1. Download and install R-Programming environment and install basic packages using `install.packages()` Command in R Programming.
2. Write a R program to take input from the user and display the values. Also print the version of R installation.
3. Learn all the basics of R-Programming (Data types, Variables, Operators etc.)
4. Write a R program to create a sequence of numbers from 20 to 50 and find the mean of numbers from 20 to 60 and sum of numbers from 51 to 61.
5. Create a function to print squares of numbers in sequence.
6. Write a program to find list of even numbers from 1 to n using R-Loops.
7. Write a R program to create three vectors a, b, c with 3 integers. Combine the three vectors to become a 3 x 3 matrix where each column represents a vector. Print the content of the matrix.
8. Write a R program to combine three arrays so that the first row of the first array is followed by the first row of the second array and then first row of the third array.
9. Write a R program to create a two-dimensional 5x3 array of sequence of even integers greater than 50.
10. Write a program to join columns and rows in a data frame using `cbind()` and `rbind()` in R.
11. Implement different String Manipulation functions in R.
12. Implement different data structures in R (Vectors, Lists, Data Frames)
13. Write a program to read a csv file and analyze the data in the file in R.
14. Create pie chart and bar chart using R.
15. Write a R program to create a simple bar plot on four subject marks.
16. Create a data set and do statistical analysis on the data using R.

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner.

Mathematical Logic (AIML205)

Credit: 3+0

Course Objectives: Objective of this course is to develop a strong foundation in logic, master proof systems and logical calculations, and analyses axiomatic theories and computability.

Syllabus

1: Propositional Logic: Syntax, Unique parsing, Semantics, Equivalences, Consequences, Calculations, Informal proofs. Normal Forms and Resolution: Clauses, CNF and DNF representations, Adequacy of Calculations, SAT, Resolution refutation, Adequacy of resolution.

2: Proof Systems: Axiomatic system PC, Adequacy of PC, Analytic tableau PT, Adequacy of PT, Compactness of PL. First Order Logic: Syntax of FL, Scope and binding, Substitutions, Semantics of FL, Quantifier laws, Equivalences, Consequences.

3: Normal Forms in FL: Calculations, Informal proofs, Prenex forms, Skolem forms, Herbrand Theorem, Skolem-Lowenheim theorem, Resolution in FL.

4: Proof Systems for FL: Axiomatic system FC, Analytic tableau FT, Adequacy of FC and FT, Compactness in FL, Axiomatic Theories: Undecidability of FL, Godel incompleteness theorems.

Course Outcomes: On completion of the course the student will be able to:

CO1: Understand propositional logic, including syntax, semantics, equivalences, and logical consequences.

CO2: Master normal forms (CNF, DNF) and resolution methods for satisfiability, refutation, and logical Calculations.

CO3: Learn first-order logic (FOL), including syntax, semantics, scope, binding, quantifier laws, and logical Transformations.

CO4: Explore proof systems, including axiomatic systems and analytic tableaux, and understand their Adequacy, compactness, and limitations in both propositional and first-order logic.

References:

1. A.Singh, Logics for Computer Science, PHI, 2004.
2. A singh, and C.Goswami,Fundamentals of Logic, ICPR New Delhi, 1998.